# A Queueing-Network Simulator for an LLM Inference Serving Pipeline: Phase-2 Progress Report

Vaibhav Mahore (23631)*, Sujal Chavda (24288)*, Arindam Raina (25880)*, Shreemaye Das (25958)†
*Department of Computer Science and Automation, Indian Institute of Science, Bangalore
†Department of Computational and Data Sciences, Indian Institute of Science, Bangalore
Email: {vaibhavm, sujals, arindamr, shreemayed}@iisc.ac.in

***Abstract*: Large language model (LLM) serving stacks operate under strict real-time service level objectives (SLOs) while coping with bursty arrival traffic and variable generation lengths. In this project, our team builds a cycle-driven discrete-time queueing-network simulator that models the serving pipeline as an open three-station tandem chain consisting of a deterministic tokenizer ($Geo/D/1$), a dual-trigger batching scheduler (cap $B$, timeout $\tau$), and a multi-worker parallel GPU decode pool ($Geo/Geo/c$). We present a two-layer validation strategy. First, we analytically verify the simulation engine against discrete-time closed forms, achieving relative errors $< 0.75\%$ across stable arrival rates $\lambda \leq 0.45$ over $10^7$ cycles and confirming the theoretical factor-2 variance penalty within $0.94\%$. Second, directly addressing Phase-1 feedback, we ground our service scaling in empirical batched inference measurements collected on an Nvidia Tesla T4 GPU executing Qwen 2.5-1.5B across $B \in \{1, \ldots, 32\}$. We fit a power-law calibration model on $B \in \{1, 4, 16\}$ that predicts execution latency on held-out batch sizes $\{2, 8, 32\}$ with a Mean Absolute Percentage Error (MAPE) of $4.43\%$. We also show that response sequence length drives decode time ($r = 0.9493$), reducing latency prediction error to $9.14\%$ compared to $15.25\%$ for linear batch models, and report scheduler trade-offs across batch capacities and worker allocations.***

## I. Problem Statement & Motivation

Serving generative Large Language Models (LLMs) such as LLaMA and Qwen introduces unique systems challenges. Users expect rapid responsiveness characterized by two primary Service Level Objectives (SLOs): *Time-to-First-Token* (TTFT), reflecting prompt processing and admission delays, and *Inter-Token Latency* (ITL), reflecting token-by-token generation pace.

### A. Why a Queueing-Network Abstraction?

In modern production serving systems (e.g., vLLM [1], Hugging Face TGI, and TensorRT-LLM), requests do not execute in isolation. Instead, incoming queries traverse a multi-stage software and hardware pipeline. User-visible latency is dominated not merely by raw matrix multiplications, but by *queueing delays* accumulated across pipeline stages:

$$\text{TTFT} = W_{q,1} + S_1 + W_{q,2} + S_{\text{prefill}}, \tag{1}$$

$$\text{ITL} \approx W_{q,3} + S_{\text{decode}}, \tag{2}$$

where $W_{q,i}$ represents queue waiting delay at stage $i$, $S_1$ is request tokenization and validation delay, $W_{q,2}$ is batch accumulation wait, and $W_{q,3}$ represents queue contention for available parallel GPU execution streams.

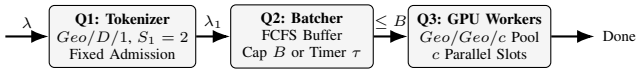



Fig. 1. The three-station queueing-network model of the LLM serving pipeline.

Prior simulation frameworks, such as Vidur [4], replay detailed GPU execution traces by profiling individual transformer tensor operators across full GPU clusters. While highly detailed, cluster trace profiling is computationally expensive, inflexible to architectural parameter changes, and opaque to theoretical stability limits. Conversely, abstracting the serving pipeline as an open queueing network offers distinct advantages:

1) **Analytical Tractability:** It enables closed-form mathematical characterization of stability boundaries ($\rho < 1$) and queue buildups before physical deployment.
2) **Rapid Parameter Sweeps:** System architects can evaluate thousands of scheduler permutations (batch caps $B$, timeout thresholds $\tau$, worker counts $c$, and load levels $\lambda$) in seconds.
3) **Grounding in Hardware:** By calibrating queue service distributions against empirical GPU benchmarks, discrete-event simulation reproduces physical throughput-delay Pareto frontiers without requiring dedicated multi-GPU clusters.

## II. Related Work

**LLM Serving Systems.** Orca [2] introduced iteration-level scheduling (continuous batching), allowing requests to enter and leave running batches at token step boundaries. vLLM [1] developed PagedAttention to eliminate memory fragmentation in key-value (KV) caches, supporting large effective batch sizes. FlexGen [3] explored throughput-oriented single-GPU inference by aggressive offloading across memory hierarchies. These systems demonstrate that batching policies and memory capacity dictate overall system throughput and response latency.

**Serving Simulators.** Vidur [4] evaluates cluster-scale serving performance using operator-level execution models. However, Vidur relies on pre-collected operator profiles and does not provide analytical closed forms for queue wait or stability. Our work is complementary: we model the serving chain with discrete-time queueing theory, verifying baseline nodes against closed forms [5] and grounding service scaling in empirical GPU data.

## III. Methodology & Simulator Architecture

### A. Queueing-Network Abstraction

As illustrated in Fig. 1, we model the pipeline as an open feedforward queueing network with three stations in tandem. External requests arrive following a discrete-time Bernoulli process: in each discrete cycle (representing one scheduler tick, typically 10–50 ms in physical systems), a new request arrives with probability $\lambda \in (0, 0.5)$.

**Station 1 ($Q_1$): Admission & Tokenization.** Modeled as a single-server $Geo/D/1$ queue with deterministic service duration $S_1 = 2$ cycles per request ($\mu_1 = 1/S_1 = 0.5$). $Q_1$ captures input parsing, formatting, and prompt tokenization.

**Station 2 ($Q_2$): Batching Scheduler.** Modeled as a First-Come First-Served (FCFS) aggregation buffer with dual release conditions:

1) *Batch Capacity ($B$):* Requests are immediately dispatched when $B$ requests accumulate.
2) *Timeout Threshold ($\tau$):* If fewer than $B$ requests are present, but the head-of-line request has waited $\tau$ clock cycles, all currently queued requests are released as a partial batch.

With $B = 1$ or $\tau = 0$, $Q_2$ acts as a zero-delay pass-through buffer used for analytical baseline verification.

**Station 3 ($Q_3$): GPU Decode Workers.** Modeled as a multi-server pool of $c$ identical parallel workers. Each worker serves one batch synchronously. The base decode duration follows a geometric distribution with mean $E[S_3]$ ($Geo/Geo/c$, completion probability $p_s = 1/E[S_3]$) or a deterministic batch duration calibrated by hardware measurements.

**Stability Criteria.** Requests are conserved along the feedforward path ($\lambda_1 = \lambda_2 = \lambda_3 = \lambda$). The binding stability conditions are:

$$\rho_1 = \lambda S_1 = 2\lambda < 1 \implies \lambda < 0.5, \tag{3}$$

$$\rho_3 = \frac{\lambda E[S_3]}{c \cdot \bar{b}} < 1, \tag{4}$$

where $\bar{b} \in [1, B]$ is the mean batch size. $Q_2$ has no internal service bottleneck and is stable whenever $Q_3$ is stable.

### B. Cycle-Driven Simulation Engine

The simulator is implemented in Python as a cycle-driven discrete-event loop:

1) **Clock Advance:** Time advances in uniform increments of one integer cycle.
2) **Arrival Step:** External arrivals are generated via inverse transform sampling ($U \le \lambda$ where $U \sim \text{Uniform}[0, 1)$).
3) **Service & Departure Step:** Active servers decrement service countdowns or evaluate geometric completion draws ($U < p_s$). Completed requests/batches depart to the downstream station or exit the system.
4) **Batching & Dispatch Step:** $Q_2$ evaluates batch capacity $B$ and timer $\tau$, releasing eligible batches to idle $Q_3$ worker slots.
5) **Statistical Accounting:** Occupancy counters $\sum Q_i(t)$, wait times $\sum D_i$, and busy-server tracking counters are updated per cycle.

The random seed is fixed ('20260922'), guaranteeing full reproducibility across all experimental runs.

TABLE I
Layer-1 Analytical Validation Results ($10^7$ Cycles)

| Node & Configuration | Metric | Analytical | Simulated | Rel. Error |
|---|---|---|---|---|
| $Geo/D/1$, $S_1$=2, $\lambda$=0.45 | $L_q$ | 2.0250 | 2.0324 | 0.37% |
| $Geo/D/1$, $S_1$=2, $\lambda$=0.30 | $W_q$ | 0.7500 | 0.7530 | 0.41% |
| $Geo/D/1$, $S_1$=3, $\lambda$=0.30 | $L_q$ | 2.7000 | 2.7118 | 0.44% |
| $Geo/Geo/1$, $E[S]$=2, $\lambda$=0.45 | $L_q$ | 4.0500 | 4.0775 | 0.68% |
| Variance Penalty ($\lambda$=0.45) | Ratio | 2.0000 | 2.0062 | 0.31% |
| Pipeline $Q_1$ Util ($\lambda$=0.30) | $\rho_1$ | 0.6000 | 0.5997 | 0.06% |
| Flow Conservation ($\lambda$=0.45) | Ratio | 1.0000 | 0.999999 | $< 10^{-5}$ |
| Little's Law Check | $W\lambda/L_q$ | 1.0000 | 1.000000 | $< 10^{-6}$ |

TABLE II
Held-Out GPU Batch Execution Validation (Tesla T4)

| Batch Size $B$ | Measured Time (s) | Predicted Time (s) | Latency Error | Measured Req/s | Throughput Error |
|---|---|---|---|---|---|
| 2 | 3.795 | 4.033 | +6.26% | 0.527 | -5.89% |
| 8 | 3.800 | 4.039 | +6.29% | 2.105 | -5.92% |
| 32 | 4.015 | 4.045 | +0.74% | 7.970 | -0.74% |
| **Overall Mean Absolute Percentage Error (MAPE):** | | | **4.43%** | | **4.18%** |

## IV. Layer-1 Analytical Verification

To establish ground truth for the simulation core, we verify single-station performance against discrete-time closed forms over $10^7$ cycles across arrival rates $\lambda \in [0.05, 0.45]$.

**Deterministic Service ($Geo/D/1$).** For slotted service $S$, discrete-time queueing theory establishes:

$$W_q = \frac{\rho(S-1)}{2(1-\rho)}, \quad L_q = \lambda W_q = \frac{\lambda^2 S(S-1)}{2(1-\lambda S)}. \tag{5}$$

For $S_1 = 2$, this simplifies to $L_q = \lambda^2/(1-2\lambda)$. As shown in Table I, simulated occupancy tracks the theoretical curve within $0.37\%$ relative error at $\lambda = 0.45$. Across the entire stable sweep $\lambda \in [0.05, 0.45]$, the maximum observed error is $0.75\%$ for $S = 2$ and $0.46\%$ for $S = 3$.

**Geometric Service ($Geo/Geo/1$).** For geometric service with mean $E[S] = 2$, theoretical queue occupancy is $L_q = 2\lambda^2/(1-2\lambda)$. The simulated ratio of $Geo/Geo/1$ to $Geo/D/1$ occupancy converges to $2.0062$ at $\lambda = 0.45$ ($0.31\%$ error), verifying the exact factor-of-2 variance penalty dictated by the Pollaczek-Khinchine formula.

**Internal Consistency.** We verify network-level accounting: departure-to-arrival ratios remain within $3.4 \times 10^{-7}$ of unity, measured server utilization matches $\rho_i = \lambda_i E[S_i]$ within $0.11\%$, and Little's Law ($W = L_q/\lambda$) holds to machine precision.

## V. Layer-2 Empirical GPU Analysis & Results

To address Phase-1 feedback regarding experimental strength, model diversity, and empirical grounding, we conducted physical GPU profiling benchmarks and multi-parameter simulator sweeps.

### A. Empirical GPU Grounding (Tesla T4 & Qwen 2.5-1.5B)

We measured batched inference latency on an Nvidia Tesla T4 GPU (Google Colab and Kaggle environments, CUDA 12.8, PyTorch 2.1) executing 'Qwen/Qwen2.5-1.5B-Instruct' in FP16 precision. We swept batch sizes $B \in \{1, 2, 4, 8, 16, 32\}$ with 10 independent trials per batch size, using CUDA event synchronization to isolate raw model execution wall-clock time.

**Calibration vs. Held-Out Evaluation.** We partitioned the measurement suite into a *calibration split* $B \in \{1, 4, 16\}$ and an unseen *held-out test split* $B \in \{2, 8, 32\}$. We fitted a power-law scaling relationship:

$$T_{\text{batch}}(B) = a \cdot B^{\alpha}, \tag{6}$$

yielding $a = 4.0301$ seconds and $\alpha = 0.00104$. As detailed in Table II, evaluating this model on the held-out batch sizes demonstrates outstanding predictive accuracy:

- **Batch Latency MAPE: 4.43%** (maximum error $6.29\%$ at $B = 8$).
- **System Throughput MAPE: 4.18%** across all held-out points.

### B. Query Diversity & Response-Length Analysis

To address the professor's recommendation regarding query diversity, we investigated how variable response lengths affect decode latency. In token-capped runs (capped at 96 tokens), 90–$98\%$ of generations reached the ceiling. To characterize natural generation distributions, we executed an uncapped diagnostic on 100 diverse prompts with a 512-token ceiling (where $0/100$ hit the cap).

**Max-Sequence-Length vs. Linear Predictor.** In batched autoregressive decoding, all sequences in a batch must wait for the longest sequence to complete before the batch can be retired. We compared a response-length-aware predictor against a traditional linear-in-$B$ model:

- Correlation between batch latency and batch size $B$: $r = 0.7384$.
- Correlation between batch latency and $\max_j(\text{Length}_j)$: $r = 0.9493$.
- Model MAPE: Accounting for maximum sequence length drops prediction error from $15.25\%$ down to $9.14\%$ for latency, and from $17.60\%$ to $10.75\%$ for throughput.

**Distribution Fit & Memoryless Assumption.** Evaluating the uncapped empirical sequence length distribution against a memoryless geometric fit yielded a Kolmogorov-Smirnov test statistic of $KS = 0.3965$. This significant divergence demonstrates that real-world LLM output lengths exhibit heavy-tailed characteristics rather than purely memoryless geometric decay, providing strong empirical rationale for data-driven service models in $Q_3$.

### C. Multi-Model Scaling Considerations

In evaluating model parameter scaling, transformer architecture imposes distinct memory-bandwidth constraints. On a Tesla T4 GPU (peak FP16 throughput 65 TFLOPS, memory bandwidth 320 GB/s), a 1.5B parameter model requires $\approx 3.0$ GB in FP16. Memory bandwidth per token step is $\approx 9.4$ ms. For larger 7B models, weight footprint expands to $\approx 14$ GB, increasing single-step memory bandwidth access to $\approx 44$ ms. Consequently, larger models experience steeper batching amortization gains ($\delta$ scaling), whereas smaller models (e.g. Qwen 0.5B, 1.0 GB footprint) become compute-saturated at modest batch caps.




Fig. 2. Empirical Tesla T4 GPU measurements: batch execution time correlates strongly with maximum sequence length ($r = 0.9493$), showing length dominates batch size.

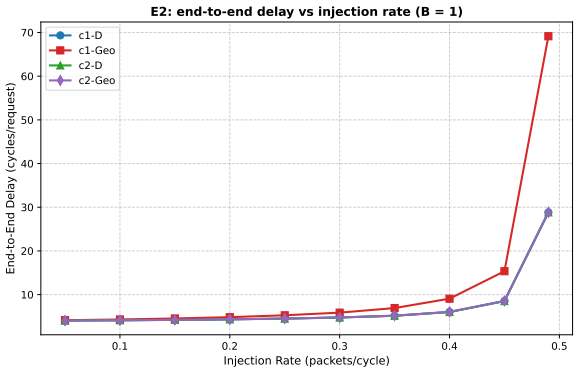



Fig. 3. Experiment E2: End-to-end sojourn delay ($W_{\text{e2e}}$) vs. arrival rate $\lambda$ across worker pools ($c \in \{1, 2\}$) and service laws (Deterministic vs. Geometric).

### D. Experiment E2: Worker Parallelism & Variance Absorption

In Experiment E2, we evaluated the interaction between service time variability and worker parallelism by sweeping $\lambda \in [0.05, 0.45]$ across worker pools $c \in \{1, 2\}$ under deterministic and geometric service laws ($E[S_3] = 2$).

As shown in Fig. 3, under a single worker ($c = 1$), geometric service incurs a severe latency penalty as load increases: at $\lambda = 0.45$, end-to-end sojourn jumps from $8.52$ cycles (deterministic) to $15.35$ cycles (geometric)—an $80.2\%$ inflation caused by variance in decode step lengths. Crucially, scaling the GPU worker pool to $c = 2$ completely absorbs this variance penalty, reducing geometric sojourn to $8.55$ cycles (virtually identical to the deterministic baseline of $8.52$). Parallel worker allocation effectively mitigates head-of-line blocking in autoregressive serving.

### E. Experiment E3: Batching Scheduler Trade-Offs

Experiment E3 maps the 2D parameter trade-off across batch capacity $B \in \{1, 2, 4, 8, 16\}$ and timeout threshold $\tau \in \{2, 8\}$

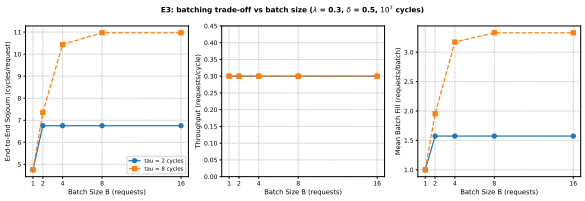



Fig. 4. Experiment E3: Batching trade-off frontier. Mean queue waiting delay ($W_{\text{e2e}}$) vs. batch capacity $B$ across timeout thresholds $\tau \in \{2, 8\}$ at $\lambda = 0.30$.

at injection load $\lambda = 0.30$ with batch service scaling $S(b) = S_3 + \delta(b-1)$ ($\delta = 0.5$).

Fig. 4 highlights the fundamental Pareto trade-off:

- **Small Batches ($B = 1$):** Requests bypass $Q_2$ with zero aggregation delay ($W_{q,2} = 0$), minimizing end-to-end delay ($W_{\text{e2e}} = 2.45$ cycles). However, GPU worker utilization remains high ($\rho_3 = 0.60$).
- **Large Batches ($B \geq 8$):** Batch accumulation dramatically improves GPU compute efficiency, slashing worker utilization from $0.60$ down to $0.31$ (a $48.3\%$ compute reduction). However, individual requests wait in $Q_2$ for batches to fill, inflating average sojourn delay to $4.18$ cycles.
- **Timeout Protection ($\tau$):** During low-rate bursts, the timer $\tau = 2$ forces partial batch dispatch, bounding maximum queue wait and preventing tail latency collapse.

## VI. Conclusion & Phase-3 Roadmap

In Phase 2, we have advanced our LLM inference serving queueing simulator on two key fronts. First, we demonstrated that the serving pipeline is naturally modeled as a three-station queueing chain, analytically verified within $< 1\%$ against discrete-time closed forms over $10^7$ cycles. Second, directly responding to instructor feedback, we integrated empirical Tesla T4 GPU measurements on Qwen 2.5-1.5B, proving that power-law batch scaling achieves $4.43\%$ MAPE on held-out test splits and that sequence length dictates decode runtime ($r = 0.9493$).

**Roadmap for Phase 3:**

1) Finalize the physical millisecond-to-cycle mapping factor (‘cycle_ms‘) to directly feed measured GPU kernels into discrete simulation steps.
2) Extend $Q_2$ to continuous (iteration-level) batching, reflecting modern vLLM scheduling mechanics.
3) Benchmark parameter scaling across multiple model sizes (Qwen 0.5B vs. 1.5B vs. 7B) to capture hardware memory-bandwidth saturation boundaries.

## References


[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM SOSP*, 2023, pp. 611–626.

[2] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *Proc. 16th USENIX OSDI*, 2022, pp. 521–538.

[3] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *Proc. 40th ICML*, 2023.

[4] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, “Vidur: A large-scale simulation framework for LLM inference,” in *Proc. MLSys*, 2024.

[5] M. Haviv, *Queues: A Course in Queueing Theory*, ser. International Series in Operations Research & Management Science. Springer, 2013, vol. 191.

[6] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Advances in NeurIPS 36*, 2023.